# Incident Response 2

## Summary

Following the initial stabilization, Phase 2 of the incident response focused on **Persistence Removal**, Privilege Escalation Mitigation, and Service Usability Restoration. The adversary transitioned from simple service disruption to complex persistence mechanisms (Windows Scheduled Tasks, Shadow Flags) and identity-based attacks (AD CS exploitation and Admin-level account creation).

---

## 1. Persistence via Malicious Scheduled Tasks (Windows/AD)

- **Detection & Evidence:** At 15:35, the **TICKETS** service experienced repeated connection timeouts. Forensics identified a suspicious Windows Scheduled Task named `automaticappupdate`. Unlike standard scripts, this task used a **Class ID (CLSID) action**, executing hidden code rather than a visible file path.
- **Analysis:** This was a high-sophistication persistence mechanism designed to rewrite malicious configurations or kill the `dotnet` process every few minutes. It was identified as the root cause of the "flapping" usability scores.
- **Mitigation:** * Executed PowerShell: `Disable-ScheduledTask -TaskName "automaticappupdate"`.
  - Cleared the rogue `python.exe` process that was hogging Port 80, returning the port to `System PID 4` control.

## 2. Privilege Escalation

- **Detection & Evidence:** Users signing up for standard ticketing accounts were being automatically granted **Domain Admin** privileges in Active Directory. "Internal Server Error 500" occurred on admin login pages due to corrupted AD Certificate Services (AD CS).
- **Analysis:** The Red Team exploited an **ESC1 misconfiguration**, using compromised templates to mint Domain Admin certificates. Furthermore, the cryptographic `Sign` function in the ticketing app was tampered with to allow any user to request signatures for "admin" subjects.
- **Mitigation:**
  - **Patching:** Hardened the `Sign` function to explicitly reject "admin" or "administrator" subjects.
  - **Infrastructure:** Restored the corrupted Certificate Authority (CA) and recovered the `adsiedit.msc` tools deleted by the adversary.
  - **Database Hardening:** Restricted the web app to **READ ONLY** on sensitive tables: `REVOKE UPDATE, DELETE ON competition_db.payment_records FROM 'flask'@'localhost'`.

## 3. Identity and Access Audit (Brute Force Detection)

- **Detection & Evidence:** At 18:50, a significant spike in unauthorized access attempts was noted.
  - **Command:** `grep "Failed password" /var/log/secure`.
  - **Result:** Identified multiple IPs (notably `68.32.254.50`) attempting to brute-force the `www` server.
- **Analysis:** The attackers attempted to regain entry after their initial PAM backdoor was closed in Phase 1.
- **Mitigation:**
  - Restarted firewalls across **AD, TICKETS, and MGMT** to flush active connections.
  - Implemented `fail2ban` on SSH to automatically block IPs exceeding three failed attempts.

## 4. Integrity Management

- **Detection & Evidence:** During flag audits at 18:25, two files with identical names were found in the same directory, one appended with a tilde (`~`).
- **Analysis:** This indicated the Red Team was using "shadowing" or temporary file swaps to confuse automated scoring scripts. Red Team flags were also discovered in `C:\Users\Administrator\` on both the AD and TICKETS servers.
- **Mitigation:**
  - Deleted all Red Team flags from administrative directories.
  - **Flag Restoration:** Compared existing hashes with the master backup; for empty flag files, the correct hash was manually re-inserted.
  - **Immutability:** Applied `chattr +i` to all Linux flag files at `/etc/` and utilized `icacls` on Windows to lock `tickets.db`.

## 5. Service Usability & Port Conflicts

- **Detection & Evidence:** MGMT HTTP (Port 5000) was refusing connections due to a "zombie" process.
- **Analysis:** A "ghost" process (PID 2508) was bound to Port 5000, preventing the patched Flask app from starting. Additionally, the `dotnet` environment was defaulting to the wrong port due to a hijacked `launchSettings.json`.
- **Mitigation:**
  - **Process Clearance:** Used `netstat -ano | findstr :5000` to identify and `taskkill /F /PID 2508` to free the port.
  - **Environment Override:** Forced production mode using `dotnet run --urls "http://0.0.0.0:80" --no-launch-profile`.
  - **Code Patch:** At 20:32, restored the "Purchase Button" functionality by re-inserting Python code deleted by the Red Team.

---

## Impact Assessment

| Intrusion | Assessment of Impact | Mitigating Measures |
|---|---|---|
| AD CS ESC1 | **Critical:** Unauthorized Domain Admin rights. | CA Restoration & Certificate template hardening. |
| CLSID Task | **High:** Persistent service flapping. | Disabled `automaticappupdate` task. |
| Port Conflicts | **Medium:** Prevented security patches from loading. | PID 2508 termination & manual port binding. |
| Shadow Flags | **Low:** Potential score suppression. | File system audit & `chattr +i` application. |

## Supporting Evidence Summary

- **Log Files:** `/var/log/secure` showing high-frequency failed password attempts from IP `68.32.254.50`.
- **PowerShell Output:** `Get-ScheduledTask` results identifying the non-standard CLSID action.
- **System State:** `netstat` output confirming PID 2508 binding Port 5000.
- **Hashes:** Integrity verification of flag `rWR0Jnr2uYTtZ1QkMhLuROWxhuxAykzTGVNtQsZxnsvSa6dg2A`.